

I want the data to be fetched In this page properly from backend but the problem is I have been trying harder with all possible methods I could! still I have tried to fetch it but I am able to fetch the data around 11- 12 times then It disappears I am not able to figure out the root cause Please find what might be the error!

Looking at your code, I don't think the problem is in the table rendering. The symptom you described:

"Data gets fetched 11–12 times and then disappears"

usually points to one of these issues:

1. API returning undefined/null intermittently

You are doing:

```
axios.get(` ${ip}/api/modules/domestic/kpis` )  
  .then((res) => {  
    setKpisData(res.data?.data);  
  })
```

and

```
axios.get(` ${ip}/api/modules/domestic/all-description` )  
  .then((res) => {  
    setDomesticTableData(res.data?.data);  
  })
```

If the backend returns:

```
{  
  "success": true,  
  "data": null  
}
```

or

```
{  
  "success": true  
}
```

even once, then:

```
setDomesticTableData(undefined);
```

After that:

```
domesticTableData.map(...)
```

fails or renders empty.

Safer:

```
setDomesticTableData(Array.isArray(res.data?.data)  
  ? res.data.data  
  : []);
```

and

```
setKpisData(Array.isArray(res.data?.data)  
  ? res.data.data  
  : []);
```

2. React Strict Mode causing double execution

If your application uses:

```
<React.StrictMode>
```

in main.jsx or index.js, then:

```
useEffect(() => {
```

```
  ...
```

```
}, []);
```

runs twice in development.

This causes:

Fetch #1

Fetch #2

and can look like data is disappearing.

Check:

```
<React.StrictMode>
```

```
  <App />
```

```
</React.StrictMode>
```

If yes, temporarily remove it and test.

3. Backend connection pool issue

Your component fetches:

```
/api/modules/domestic/kpis
```

```
/api/modules/domestic/all-description
```

only once.

So if data disappears after 11–12 successful loads across page refreshes, the problem may actually be backend-side:

Possible causes

- DB connection not released
- Sequelize connection leak
- MSSQL connection leak
- Oracle cursor exhaustion
- Pool max reached

Example:

```
const connection = await pool.getConnection();
```

```
// query
```

```
// missing
```

```
connection.release();
```

After several requests:

Request 1 ✓

Request 2 ✓

...

Request 11 ✓

Request 12 ✗

This matches your symptom exactly.

4. IP changing unexpectedly

You store:

```
localStorage.setItem("SERVER_IP", ipAddress);
```

and later use:

```
const cachedIp = localStorage.getItem("SERVER_IP");
```

Add:

```
console.log("SERVER IP =", cachedIp);
```

before:

```
fetchDashboardData(cachedIp);
```

I have seen situations where:

172.195.121.61

becomes:

undefined

or

""

in localStorage after refresh.

Then requests go to:

undefined/api/modules/domestic/kpis

and silently fail.

5. Component unmounting and remounting

You have:

```
let isMounted = true;
```

which is good.

But if parent routes keep remounting:

```
<MonthlyDomestic />
```

then every navigation creates:

New useEffect

New API call

New state reset

Check with:

```
useEffect(() => {
```

```
  console.log("MonthlyDomestic Mounted");
```

```
return () => {  
  console.log("MonthlyDomestic Unmounted");  
};  
}, []);
```

If you see:

Mounted

Unmounted

Mounted

Unmounted

continuously, the problem is routing.

6. Backend sending cached empty response

Add logs:

```
axios.get(`${ip}/api/modules/domestic/all-description`)  
.then((res) => {  
  console.log("API RESPONSE", res.data);  
  setDomesticTableData(res.data?.data || []);  
});
```

Watch Network tab.

If you see:

```
{  
  "data": []  
}
```

after the 11th/12th call,

then frontend is innocent.

The backend is returning empty data.

7. Biggest red flag in your code

You initialize with mock data:

```
const [domesticTableData, setDomesticTableData] =  
  useState(mockData.domesticLeadsTableData);
```

Then later:

```
setDomesticTableData(res.data?.data);
```

If API fails:

```
.catch((err) => console.error(err));
```

you do NOT restore mock data.

So the table can suddenly become:

undefined

or

```
[]
```

and appear to "disappear".

Change to:

axios

```
axios.get(`${ip}/api/modules/domestic/all-description`)
  .then((res) => {
    setDomesticTableData(
      Array.isArray(res.data?.data)
        ? res.data.data
        : mockData.domesticLeadsTableData
    );
  })
  .catch((err) => {
    console.error(err);
    setDomesticTableData(mockData.domesticLeadsTableData);
  });
```

What I need to find the exact root cause

Please upload or paste:

1. Backend code for:
 - o /api/modules/domestic/kpis
 - o /api/modules/domestic/all-description
2. Browser Console errors after the data disappears.
3. Network tab response of:
 - o /api/modules/domestic/all-description

The issue is very likely on the backend side (connection leak, pool exhaustion, or API returning empty data) rather than in this React page itself. The frontend code you've shared does not contain anything that would naturally stop working after exactly 11–12 successful fetches.

Explain each solution in steps In detail

Let's debug this systematically. Based on the code you uploaded, the React page itself looks mostly correct. The fact that:

Data loads successfully 10–12 times and then disappears

usually indicates that something external (backend, API response, connection pool, routing, caching, etc.) is failing after some time.

Step 1: Verify Whether the API is Actually Returning Data

Why?

Your frontend only displays whatever the backend sends.

Currently:

```
axios.get(`${ip}/api/modules/domestic/all-description`)
  .then((res) => {
```

```
    setDomesticTableData(res.data?.data);  
  })
```

You never verify what the API actually returned.

Add Logging

Modify:

```
axios.get(` ${ip}/api/modules/domestic/all-description` )  
  .then((res) => {  
    setDomesticTableData(res.data?.data);  
  })
```

to:

```
axios.get(` ${ip}/api/modules/domestic/all-description` )  
  .then((res) => {  
  
    console.log("API Response:");  
    console.log(res.data);  
  
    console.log("Data Length:");  
    console.log(res.data?.data?.length);  
  
    setDomesticTableData(res.data?.data);  
  })
```

Open Browser Console

Press:

F12

Go to:

Console Tab

Watch the logs.

Scenario A

Before disappearing:

Data Length: 25

After disappearing:

Data Length: 0

or

undefined

Then:

☒ Backend is returning bad data.

Scenario B

Still shows:

Data Length: 25

but table disappears.

Then:

✅ Frontend rendering issue.

Step 2: Verify Network Response

Why?

Sometimes console logs don't show enough.

Open:

F12

→ Network

Refresh page.

Look for:

/api/modules/domestic/all-description

Click it.

Check

Response Tab

See whether:

```
{  
  "data": [...]  
}
```

or

```
{  
  "data": []  
}
```

Status Code

Check:

200

or

500

or

502

or

504

Result

If response becomes:

```
{  
  "data": []  
}
```

after 10–12 loads:

Frontend is innocent.

Backend is guilty.

Step 3: Check Whether Component is Re-Mounting

This is very common.

Add:

```
useEffect(() => {  
  
  console.log("MonthlyDomestic Mounted");  
  
  return () => {  
    console.log("MonthlyDomestic Unmounted");  
  };  
  
}, []);
```

Watch Console

If you see:

Mounted

Unmounted

Mounted

Unmounted

Mounted

Unmounted

every time you switch pages,
your component is constantly being recreated.

Why It Matters

When recreated:

const [domesticTableData, setDomesticTableData]
gets reset.

API gets called again.

State disappears.

Step 4: Verify React Strict Mode

Open:

main.jsx

or

index.js

Look for:

```
<React.StrictMode>
  <App />
</React.StrictMode>
```

Why?

React intentionally executes effects twice in development.

Example:

Fetch #1

Fetch #2

for every page load.

This often looks like:

Data loaded

Data disappeared

Data loaded

Test

Temporarily change:

```
ReactDOM.createRoot(document.getElementById('root')).render(
  <App />
);
```

without:

```
<React.StrictMode>
```

Run again.

Step 5: Verify localStorage IP

You are doing:

```
localStorage.setItem("SERVER_IP", ipAddress);
```

and later:

```
const cachedIp = localStorage.getItem("SERVER_IP");
```

Add Logging

Before:

```
fetchDashboardData(cachedIp);
```

add:

```
console.log("Cached IP:", cachedIp);
```

Expected

Cached IP: http://172.195.xxx.xxx

Bad Result

Cached IP: null

or

Cached IP: undefined

or

Cached IP:

Then requests become:

undefined/api/modules/domestic/kpis

which fail.

Step 6: Add Proper Error Logging

Currently:

```
.catch((err) => console.error(err));
```

is not enough.

Replace with:

```
.catch((err) => {
```

```
    console.log("ERROR OCCURRED");
```

```
    console.log("Message:", err.message);
```

```
    console.log("Status:", err.response?.status);
```

```
    console.log("Response:", err.response?.data);
```

```
});
```

Example Output

Status: 500

means backend crashed.

or

Status: 401

token issue.

or

Status: 504

timeout.

Step 7: Check Database Connection Pool

This is the MOST LIKELY cause if it works 10–12 times and then stops.

Example Backend

```
router.get("/all-description", async (req,res)=>{  
  
  const connection = await pool.getConnection();  
  
  const data = await connection.query(sql);  
  
  res.json(data);  
  
});
```

Problem

Connection never released.

Missing:

```
connection.release();
```

What Happens

Request 1

Connection #1 allocated

Request 2

Connection #2 allocated

Request 3

Connection #3 allocated

...

Request 11

Connection #11 allocated

Pool exhausted.

Request 12

No connection available

Data disappears.

Check Backend Logs

Look for:

Connection pool exhausted

or

Too many connections

or

Timeout acquiring connection

Step 8: Protect State From Bad Responses

Current:

```
setDomesticTableData(res.data?.data);
```

Dangerous.

Use:

```
setDomesticTableData(  
  Array.isArray(res.data?.data)  
    ? res.data.data  
    : []  
);
```

Even better:

```
setDomesticTableData(  
  Array.isArray(res.data?.data)  
    ? res.data.data  
    : mockData.domesticLeadsTableData  
);
```

Step 9: Detect Infinite API Calls

Add:

```
console.count("API CALL");
```

inside:

```
fetchDashboardData()
```

Example:

```
const fetchDashboardData = (ip) => {  
  
  console.count("API CALL");  
  
  axios.get(...)  
}
```

Watch Console

Good:

```
API CALL: 1
```

Bad:

```
API CALL: 1
```

```
API CALL: 2
```

```
API CALL: 3
```

```
API CALL: 4
```

...

API CALL: 50

Then some parent component is causing repeated renders.

My Ranking of Most Likely Causes

Based on your exact symptom:

Probability	Cause
★★★★★	Backend DB connection leak / pool exhaustion
★★★★	API returning empty array after some requests
★★★	Component remounting repeatedly
★★★	React StrictMode double execution
★★	Invalid SERVER_IP from localStorage
★	Table rendering issue

Since you said:

"I can fetch around 11–12 times then it disappears"

that pattern is extremely characteristic of a database connection pool limit (commonly 10, 15, or 20 connections) being exhausted on the backend.

To confirm this with certainty, I would need to see the backend code for:

/api/modules/domestic/kpis

/api/modules/domestic/all-description

especially the database query and connection handling logic.